BSc (Hons) in **Information Technology Specialized in AI**

Faculty of Computing

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Practical 03

> **Objective & Lecture Mapping:** In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

> Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

> You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent`.
2. Import required structures: `from collections import deque` and `import heapq`.
3. **BFS:** Create `def bfs_search(...)`. Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)`. Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)`. Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

> The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__`, add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'`.
2. In `sense_and_act(self, percept)`, check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']`, execute the search method matching `self.active_algo`, and store the resulting sequence of actions in `self.plan`.
4. Return the first action from the plan using `return self.plan.pop(0)`.
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

> Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

State Space: This is the set of all possible configurations or locations in the environment. In your grid world, the state space consists of every unique (x,y) coordinate that the agent can occupy. It is a physical "map" and often contains cycles .
Search Tree: This is a conceptual representation of the paths being explored by the algorithm. Each node in the tree represents a sequence of actions starting from the initial state.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

In graph search theory, the reached (or visited) set solves the problem of redundant paths and cycles within the state space.
By tracking which coordinates have already been explored, the algorithm prevents itself from re-processing the same location multiple times. This is the key mechanism that converts a Tree Search into a Graph Search.
Without a reached set, a DFS agent would likely enter an infinite loop. Because DFS prioritizes depth, it might move from position A to B, then immediately move back to A, then back to B, and so on forever. Since it has no memory of being at A before, it will never stop oscillating or moving in circles, causing the search to never terminate and eventually lead to a crash.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

BFS (Breadth-First Search) is guaranteed to be optimal in this grid because it explores nodes in order of their distance from the start (level by level). Since every move in the grid has an equal cost of 1, BFS will always find the shortest path and the first time it reaches the goal, it is guaranteed to have used the minimum number of steps.

DFS (Depth-First Search), on the other hand, prioritizes depth over proximity. It dives as deep as possible along a single branch of the search tree before

backtracking. It does not "look around" for shorter alternatives; it simply returns the very first path it happens to find. Because the order of exploration is arbitrary, DFS will often follow a long, winding path to a goal that might have been only a few tiles away, resulting in highly suboptimal routes.

4. (Evaluate) If we scaled `visual_grid_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

The memory bottleneck contrast between BFS and DFS is defined by their space complexity.
BFS (Breadth-First Search) / UCS: Has an exponential space complexity of O(b^d). It must store every node in a widening the queue. In a massive grid, this frontier grows so large that it will exhaust the system's RAM and crash before reaching a distant goal.
DFS (Depth-First Search): Has a linear space complexity of O(bm)
It only needs to store the current path from the start to the leaf node. This requires significantly less memory, allowing it to operate in large grids without crashing, though it will likely return a non-optimal, winding path.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation

Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.



FACULTY OF COMPUTING